

Akamai SIEM to Coralogix — Docker Compose Deployment

Collect Akamai SIEM events and forward them to Coralogix for the Akamai WAF extension using a single Docker Compose service.

1. What this does

```
Akamai SIEM API
  |
  v
akamai-siem pull --emit-datastream    (writes daily NDJSON, saves offset)
  |
  v
akamai-siem send                      (Coralogix /logs/v1/singles)
  |
  v
Coralogix app <application> / subsystem <subsystem>
```

Native send posts each event in the Coralogix text field, which is the shape the Akamai WAF extension parses. No OpenTelemetry collector is required.

2. Requirements

- Docker with Compose v2 (or Podman + native podman-compose).
- Akamai SIEM configuration ID.
- Akamai .edgerc with SIEM read access.
- Coralogix key JSON containing apiKey.keyValue with send-data permission.
- linux/amd64 container support (the binary is linux/amd64).

3. Repository files

File	Purpose
akamai-siem-linux-amd64	Prebuilt CLI binary (linux/amd64)
SHA256SUMS	Binary checksum
Dockerfile	Runtime image (binary only, non-root, no secrets)
compose.yaml	Deployment (pull + send loop)
.env.example	Non-secret settings + credential file paths

4. Verify the binary

```
sha256sum -c SHA256SUMS
# akamai-siem-linux-amd64: OK
```

5. Prepare credentials

```
chmod 600 /path/to/.edgerc
chmod 600 /path/to/coralogix-key.json
```

Keep both files on the host and never commit them.

6. Configure .env

```
cp .env.example .env
# edit: AKAMAI_CONFIG_ID, AKAMAI_EDGERC_FILE,
#       CORALOGIX_DOMAIN, CORALOGIX_APPLICATION,
#       CORALOGIX_SUBSYSTEM, CORALOGIX_KEY_FILE
```

.env holds non-secret settings plus the paths to the two credential files. Never put the contents of .edgerc or a Coralogix key into it.

7. Create runtime directories

```
mkdir -p runtime/state runtime/send-state runtime/out
chown -R 100:101 runtime
chmod 700 runtime/state runtime/send-state
chmod 750 runtime/out
```

The container runs as UID 100 / GID 101 (akamai).

Host dir	Container path	Contents
runtime/state	/var/lib/akamai/state	Akamai pagination offset
runtime/send-state	/var/lib/akamai/send-state	Coralogix byte offsets + last-ok
runtime/out	/var/lib/akamai/out	daily events-YYYYMMDD.ndjson

8. Start

```
docker compose --env-file .env up -d --build
```

9. Logs and health

```
docker compose --env-file .env ps          # expect: healthy
docker compose --env-file .env logs -f akamai-siem
```

Healthy cycle log output:

```
done: pages=... .. written=... out=/var/lib/akamai/out/events-YYYYMMDD.ndjson
sent: format=datastream events=... file=...
send complete: events=... files=...
cycle ok; sleeping 600s
```

10. Verify delivery with local cx (optional)

Compose itself does not use the cx CLI. To confirm records are visible, query with a cx profile for the same tenant/region:

```
cx logs "source logs | filter $1.applicationname == '<application>' | filter
$1.subsystemname == '<subsystem>' | limit 10" \
--start now-30m --end now --tier frequent -o json --read-only
```

- Delivery — send requests succeed (see logs).
- Visibility — records appear in the query.
- Parsing — `$d.cx_security.*` fields materialize (deployed Akamai WAF extension).
- Alerts — the extension's alerts fire on parsed events.

11. Restart, stop, upgrade

```
docker compose --env-file .env restart
docker compose --env-file .env down          # stop; state preserved
```

Upgrade:

```
docker compose --env-file .env down
# replace akamai-siem-linux-amd64, re-run sha256sum -c SHA256SUMS
docker compose --env-file .env up -d --build
```

State is on the host, so restarts resume without re-sending acknowledged data.

12. Troubleshooting

Symptom	Likely cause	Action
Restarts every RETRY_DELAY	Pull or send failing	Check docker compose logs
401/403 on send	Key lacks access or wrong team	Rotate/scope key; check domain
Permission denied on out/state	Bind mounts not owned by 100:101	Re-run section 7 chown
416 offset expired	Saved cursor too old	CLI resets to from=now-12h automatically
No such image ... image not known	docker-compose shim on Podman	Use native podman-compose

13. Security notes

- Image contains only the binary; runs as a non-root user.
- Credentials mounted read-only as Docker secrets, never baked in.
- `.env`, `.edgerc`, and the Coralogix key must never be committed.
- No customer event data, tenant IDs, or API keys are included.

14. Supported architecture

linux/amd64. The prebuilt binary and image target that platform, enforced via platform:
linux/amd64 in compose.yaml.